

ENSEA

Beyond Engineering

REAL-TIME OPERATING SYSTEMS

Lab – Porting a Shell

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

The story

- An intern wrote a **shell**: type a command in a terminal, a C function runs on the board
- It runs in bare metal, in a blocking loop...
- ...and it is **full of bugs**
- It is available in the `td6` project

Your job

1. Use it (guided)
2. Understand it (guided)
3. Port it to bidule, in our RTOS architecture – from a specification only: the design is yours

ENSEA
Beyond Engineering

Today

1 The User's Point of View

2 The Developer's Point of View

3 Porting: the Specification

4 One More Thing

ENSEA
Beyond Engineering

First contact

1. Flash the `td6` project on the board
2. Start a terminal emulator (115200 bauds)
3. Press `h`, then `Enter`: the list of commands
4. Run the dummy function: `f`

```
===== Monsieur Shell v0.2 =====
> h
:h
h: Help
f: Une fonction bidon
> f
:f
Bongeourre-han, je suis
une fonction nulle-han
...
```

ENSEA
Beyond Engineering

Your first commands

1. Modify the dummy function so that it prints the list of its arguments

```
> f one two 3
argc = 4
argv[0] = f
argv[1] = one
...
```

2. Add a command that prints the **sum** of all its arguments

```
> a 1 2 39
42
```

ENSEA
Beyond Engineering

Reading the code (1/2)

1. List the functions available to the user of the module (its API). List the hidden ones.
2. How is the serial link to write on chosen?
3. List the static variables and structures. What does the keyword `static` mean, for a variable and for a function?
4. How do you add a function callable from the shell?

ENSEA
Beyond Engineering

Reading the code (2/2)

5. In which function are the characters read?
6. What happens when you press `Enter`?
7. How does deleting a character work?
8. To which function is the command string passed?
9. In that function, how do we end up calling the dummy function?

ENSEA
Beyond Engineering

What the shell must do on bidule

- Port the shell to bidule, on top of the `td5` project
- From the shell, change the state of the LEDs and of the Neopixel:

```
> l 1      # toggles LED 1
> l 3      # toggles LED 3
> l 4      # prints an error message: there is no LED 4
> u 300    # sets the hue to 300
> s 100    # sets the saturation to 100
> v 300    # prints an error: the value cannot exceed 255
```

- The commands act on the hardware **and** change the state of the APP
  - `u 300` changes the colour of the Neopixel **and** the value shown in the menu

ENSEA
Beyond Engineering

Design requirements

- The shell follows an **object** model: we can create several instances (UART, USB, Bluetooth...)
- The shell runs in the context of the **APP active object**
  - So it must **never block**
- The characters are read by the **BSP**, **without polling**
- The BSP sends the characters to the APP as **events**, through its queue
- Simplification: the output may use `printf`
  - Yet `printf` blocks (UART in polling) and is shared by all the instances: it breaks run-to-completion *and* the object model
  - Same kind of problem as the display – we leave it aside for now

No more guidance

How the code is organised is up to you. Use what we built during the course: BSP / APP separation, objects, events, active object, run-to-completion, interrupts and FreeRTOS.

ENSEA
Beyond Engineering

Before writing any code

1. List the steps to carry out
2. For each step, list the files to modify – or to create
3. Draw how a character travels, from the UART to the function called by the shell

Then

Implement, one step at a time, and test after each one.

The intern was me!

- This code was written a long time ago – and naively, kept on purpose
- Do not hesitate: **list all the bugs** you find, and **fix them**

ENSEA
Beyond Engineering

Bonus

Get rid of `printf`

Solve the output problem: the shell must never block, and each instance must write on its own link.